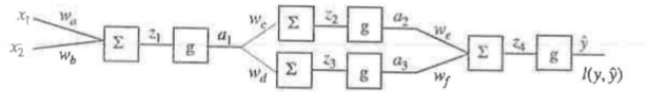


Backpropagation

Chain rule on a computation graph For each parameter w , multiply local derivatives along every path from w to the loss ℓ .



General rules

- **Chain (series):** $\frac{\partial \ell}{\partial w} = \frac{\partial \ell}{\partial y} \cdot \frac{\partial y}{\partial z} \dots \frac{\partial u}{\partial w}$
- **Linear node** $z = \sum_i w_i x_i$: $\frac{\partial z}{\partial w_i} = x_i, \quad \frac{\partial z}{\partial x_i} = w_i$
- **Activation** $a = g(z)$: $\frac{\partial a}{\partial z} = g'(z)$

Worked example Network: Output-side weight (no branching):

$$\frac{\partial \ell}{\partial w_e} = \frac{\partial \ell}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_4} \cdot \frac{\partial z_4}{\partial w_e}$$
$$\frac{\partial \ell}{\partial w_c} = \frac{\partial \ell}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_4} \cdot \frac{\partial z_4}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_c}$$

Input weight w_a (here a_1 branches into z_2 and z_3 , so two paths sum) at a and anything past it:

$$\frac{\partial \ell}{\partial w_a} = \frac{\partial \ell}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_4} \cdot \left[\frac{\partial z_4}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} + \frac{\partial z_4}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_1} \right] \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_a}$$

where each $\frac{\partial \ell}{\partial a_k}$ continues forward through $z_4, \hat{y}, \ell$.

ReLU vs. Sigmoid

Definitions $\sigma(z) = \frac{1}{1+e^{-z}}, \sigma'(z) = \sigma(z)(1 - \sigma(z)) \in (0, 0.25]$.

$\text{ReLU}(z) = \max(0, z), \text{ReLU}'(z) = \mathbb{I}[z > 0] \in \{0, 1\}$.

Why ReLU for hidden layers

- **Vanishing gradients:** sigmoid saturates (σ' tiny when $|z|$ large). Deep nets multiply many $\sigma' < 1$ factors $\Rightarrow$ early-layer gradients vanish. ReLU's gradient is 1 in the active region, so signal flows.
- **Sparsity:** ReLU outputs exact zeros for $z < 0 \Rightarrow$ sparser activations, often easier optimization.
- **Cheap:** just a max, no exponentials.
- **Not zero-centered (a downside)**, fixed by Leaky/ELU variants. "Dying ReLU": units stuck at $z < 0$ get 0 gradient forever.

L1 / L2 Regularization

Penalties added to loss:

$$L_2 : \frac{\lambda}{2} \|w\|_2^2 = \frac{\lambda}{2} \sum_i w_i^2 \quad L_1 : \lambda \|w\|_1 = \lambda \sum_i |w_i|$$

Vector derivatives

$$\nabla_w \frac{1}{2} \|w\|_2^2 = w \quad \nabla_w \|w\|_1 = \text{sign}(w)$$

(componentwise; subgradient at 0). Update with L_2 : $w \leftarrow w - \alpha(\nabla \ell + \lambda w) = (1 - \alpha\lambda)w - \alpha \nabla \ell$ - "weight decay."

Effects

- L_2 : shrinks all weights smoothly toward 0, never exactly 0. Penalizes large weights heavily (quadratic).
- L_1 : pushes weights to *exactly* 0 $\Rightarrow$ sparsity / feature selection. Penalty grows linearly.

Unit balls in $\mathbb{R}^2$ $\{x : \|x\| = c\}$:

- L_2 : **circle** of radius c .

• L_1 : **diamond** (square rotated 45°), vertices at $(\pm c, 0), (0, \pm c)$. Sparsity intuition: the L_1 diamond has corners on the axes, so the level set of the loss tangent to the constraint ball is most likely to touch at a corner $\Rightarrow$ a coordinate is exactly 0.

Optimizers

Initialize with uniform variance to prevent exploding gradient

Let $g_t = \nabla_w \ell(w_t)$, learning rate α .

SGD $w_{t+1} = w_t - \alpha g_t$

Momentum (heavy ball) accumulate exponentially-decaying avg of past gradients; dampens oscillation across ravines, accelerates along consistent directions.

$$v_{t+1} = \beta v_t + g_t, \quad w_{t+1} = w_t - \alpha v_{t+1}$$

Typical $\beta = 0.9$. Equivalently: $v \leftarrow \beta v + (1 - \beta)g$ form (rescaled).

Nesterov (NAG) "look ahead" before computing gradient:

$$v_{t+1} = \beta v_t + \nabla \ell(w_t - \alpha \beta v_t), \quad w_{t+1} = w_t - \alpha v_{t+1}$$

Reduces overshoot vs. plain momentum.

AdaGrad per-parameter LR; divides by $\sqrt{\text{sum of squared past gradients}}$.

Good for sparse features; LR shrinks monotonically (eventually too small).

$$G_t = G_{t-1} + g_t^2, \quad w_{t+1} = w_t - \frac{\alpha}{\sqrt{G_t} + \epsilon} g_t$$

RMSProp fixes AdaGrad's decaying LR by using exponential moving average of g^2 :

$$E_t = \rho E_{t-1} + (1 - \rho)g_t^2, \quad w_{t+1} = w_t - \frac{\alpha}{\sqrt{E_t} + \epsilon} g_t$$

Typical $\rho = 0.9$.

Adam momentum + RMSProp, with bias correction:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$$
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$
$$w_{t+1} = w_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Defaults: $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$. AdamW decouples weight decay: $w \leftarrow w - \alpha(\hat{m}/(\sqrt{\hat{v}} + \epsilon) + \lambda w)$.

Convolution / Pooling Dimensions

Conv output spatial size input $n \times n$, filter $f \times f$, stride s , channels C , padding p , and K output filters, so a convolution layer of $f \times f \times C \times K$; produces output $n_{out} \times n_{out} \times K$

$$n_{out} = \left\lfloor \frac{n + 2p - f}{s} \right\rfloor + 1$$

Conv parameter count input has C_{in} channels, layer has C_{out} filters of size $f \times f$:

$$\#weights = f \cdot f \cdot C_{in} \cdot C_{out}$$

$$\#params = f \cdot f \cdot C_{in} \cdot C_{out} + C_{out} \text{ (biases)}$$

Single output channel ($C_{out} = 1$): $\#weights = f^2 C_{in}$, $\#params = f^2 C_{in} + 1$.

Single-channel input *and* output: $\#weights = f^2$, $\#params = f^2 + 1$. Max pool 0 learnable parameters FC layer from flat d inputs to N outputs: $\#params = dN + N$.

BatchNorm per channel: $2C$ learnable $(\gamma, \beta) + 2C$ running stats (non-learnable). Goal is to mitigate interal covariance, leading to faster convergence

Padding finder for "same padding" To find padding given you want the output size to be equal to input size, set size formula equal to n and solve for p .

Inception Modules Parallel paths with different receptive field sizes and operations to capture sparse patterns of correlations in the stack of feature maps; Use 1x1 convolutions for dimensionality reduction before expensive convolutions; May be useful to increase # of parallel sets inside block

Object Detection

Confusion-matrix metrics

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

Precision: "of predicted positives, how many were correct." Recall: "of actual positives, how many did we find."

Intersection over Union (IoU) for predicted box A and ground-truth box B :

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Detection counts as TP if $\text{IoU} \geq \text{threshold}$ (commonly 0.5) *and* class matches.

Pipeline (mAP) for each class: sort predictions by confidence, sweep threshold to get precision-recall curve, AP = area under curve. mAP = mean over classes (sometimes also over IoU thresholds, e.g. COCO uses 0.5:0.05:0.95).

R-CNN Regions w CNN features, extract region proposals, compute cnn features, linear classification with SVM. But this isn't an end-to-end system, with different training regimes for each of the different losses (log loss for softmax, hinge loss for svms, least squares for bounding-box). Also slow! Fast R-CNNs Convnet the whole thing, propose regions, region-of-interest pool (pool together regions) - predicting probs for $c+1$ classes (0 is background) and drawing c bounding boxes, FF then softmax and bounding box. Entire task is trainable at once! Speeds up CNN computation - instead of once per proposal, it's over the entire image Only possible disadvantage: downsampling may hurt detection for small objects. Faster R-CNN Speed up generating region proposals through a region proposal network, Slide a small window (3x3) over the conv5 layer, Predict object/no object, Regress bounding box coordinates with reference to anchors (3 scales x 3 aspect ratios)

RNNs and LSTMs

Vanilla RNN hidden state recurrence:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b), \quad y_t = W_{hy}h_t$$

Same weights at every timestep (parameter sharing). Trained with backprop through time (BPTT): unroll, apply chain rule.

Problem repeated multiplication by W_{hh} over many steps causes **vanishing** (eigenvalues < 1) or **exploding** (> 1) gradients $\Rightarrow$ can't learn long-range dependencies. Exploding: clip gradients. Vanishing: use gates.

LSTM adds a cell state c_t with additive updates (gradient highway). Three gates control flow:

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) && \text{forget} \\ i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) && \text{input} \\ \tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c) && \text{candidate} \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{cell} \\ o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) && \text{output} \\ h_t &= o_t \odot \tanh(c_t) \end{aligned}$$

Forget gate of 1 + input of 0 $\Rightarrow c_t = c_{t-1}$ exactly (no decay), so gradients pass unchanged.

GRU simpler: combines forget+input into one gate, merges h and c . Fewer params, often comparable.

Attention & Transformers

Scaled dot-product attention given queries $W_q, W_k, W_v \in \mathbb{R}^{x \times d_k}, X \in \mathbb{R}^{n \times x}, Q, K, V, Z \in \mathbb{R}^{n \times d_k}$, where $Q = XW_q$ and so on, Q = decoder output, K, V = encoder output, so if encoder outputs 4x512 and decoder 2x512, $Q = 2 \times 64, K=V=4 \times 64$:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \in \mathbb{R}^{n \times d_k}$$

$\sqrt{d_k}$ prevents dot products from growing large $\Rightarrow$ softmax saturating $\Rightarrow$ vanishing gradients.

Self-attention Q, K, V all from same input X : $Q = XW^Q, K = XW^K, V = XW^V$. Each token attends to every other.

Multi-head run h attention ops in parallel with different projections, concat, project:

$$\text{MHA}(X) = [\text{head}_1; \dots; \text{head}_h]W^O$$

Lets the model attend to different subspaces / relations simultaneously.

So with h heads, $Z_i \in \mathbb{R}^{n \times d_k}, Z \in \mathbb{R}^{n \times h d_k}, W_o \in \mathbb{R}^{x \times x h}$

Transformer block

- Multi-head self-attention $\rightarrow$ residual + LayerNorm
- Position-wise feed-forward (2 linear layers w/ ReLU/GELU) $\rightarrow$ residual + LayerNorm

Positional encoding attention is permutation-equivariant; add sinusoidal or learned positional vectors to embeddings so order matters.

What teli did: given position $p, i : 0 - 255$, return matrix $\text{pos}(p, 2i), \text{pos}(p, 2i + 1)$, where $\text{pos}(p, 2i) = \sin(p/(100000^{(2i/512)}))$, $\text{pos}(p, 2i+1) = \cos(p/(100000^{(2i + 1/512)}))$, produces matrix of size 512. Using 2 words as context produces $Z \in \mathbb{R}^{2 \times 512}$. Note that the entire block doesn't change X dims or anything else like that

Masking decoder self-attention masks future positions ($-\infty$ before softmax) so token t only attends to $\leq t$.

Why transformers beat RNNs

- Parallelizable across sequence (no sequential $h_{t-1} \rightarrow h_t$).

- $O(1)$ path length between any two positions (vs. $O(n)$ for RNN) $\Rightarrow$ long-range deps easier.
- Cost: $O(n^2 d)$ per layer in sequence length.

Worked Examples (CMSC 422 Final F25)

P2. Backprop on the 2-D-input network above $\frac{\partial}{\partial w_c}(\frac{\lambda}{2}\|w\|^2) = \lambda w_c$, so

$$w_c \leftarrow w_c - \alpha \left(\frac{\partial \ell}{\partial w_c} + \lambda w_c \right)$$

P3. CNN shapes / params Fill activation shapes (H, W, C) and # params per layer. Input $32 \times 32 \times 3$. CONVx-N = conv with N filters of size $x \times x$; POOL-n = $n \times n$ max-pool, stride n , padding 0. Default $p = 1, s = 1$ for conv unless stated.

CONV3-N with $p = 1, s = 1$ preserves H, W and outputs N channels. POOL-2 halves H, W .

Layer	Shape	# params
Input	(32, 32, 3)	0
CONV3-8	(32, 32, 8)	$3 \cdot 3 \cdot 3 \cdot 8 + 8 = 224$
LeakyReLU	(32, 32, 8)	0
POOL-2	(16, 16, 8)	0
BatchNorm	(16, 16, 8)	$2 \cdot 8 = 16$
CONV3-16	(16, 16, 16)	$3 \cdot 3 \cdot 8 \cdot 16 + 16 = 1168$
LeakyReLU	(16, 16, 16)	0
POOL-2	(8, 8, 16)	0
FLATTEN	(1024,)	0
FC-10	(10,)	$1024 \cdot 10 + 10 = 10250$

P4. Softmax $\rightarrow$ binary logistic Softmax: $P(y=c|x) = \frac{\exp(w_c^\top x)}{\sum_{d \in C} \exp(w_d^\top x)}$. $P(Y=0|x)$. Ways to reduce underfitting

Increase number of neurons in hidden layers to represent more complex patterns, reduce dropout rate Ways to reduce overfitting Introduce regularization to punish large gradients/weights, use dropout (also has the side bonus of making your network more efficient since less of it is being used), Param calculation For FF weights are always output by input, so for a 20input- i 50- i 100- i 3 output network, there are $(20 \cdot 50 + 50) + (50 \cdot 100 + 100) + (100 \cdot 3 + 3)$ parameters used in total, including biases.

Conv calculations

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \end{bmatrix}, K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}, P = 1, S = 2$$

$$X_{\text{pad}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 4 & 0 \\ 0 & 5 & 6 & 7 & 8 & 0 \\ 0 & 9 & 1 & 2 & 3 & 0 \\ 0 & 4 & 5 & 6 & 7 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Position (0,0) — kernel covers X_{pad} rows 0–2, cols 0–2:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 2 \\ 0 & 5 & 6 \end{bmatrix} \odot \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

$$= 0 + 0 - 0 + 0 + 0 - 2 + 0 + 0 - 6 = -8$$

Position (0,1) — slide right by stride 2; rows 0–2, cols 2–4:

$$\begin{bmatrix} 0 & 0 & 0 \\ 2 & 3 & 4 \\ 6 & 7 & 8 \end{bmatrix} \odot K$$

$$= 0 + 0 - 0 + 2 + 0 - 4 + 6 + 0 - 8 = -4$$

Position (1,0) — slide down by stride 2; rows 2–4, cols 0–2:

$$\begin{bmatrix} 0 & 5 & 6 \\ 0 & 9 & 1 \\ 0 & 4 & 5 \end{bmatrix} \odot K$$

$$= 0 + 0 - 6 + 0 + 0 - 1 + 0 + 0 - 5 = -12$$

Position (1,1) — rows 2–4, cols 2–4:

$$\begin{bmatrix} 6 & 7 & 8 \\ 1 & 2 & 3 \\ 5 & 6 & 7 \end{bmatrix} \odot K$$

$$= 6 + 0 - 8 + 1 + 0 - 3 + 5 + 0 - 7 = -6$$

$$Y = \begin{bmatrix} -8 & -4 \\ -12 & -6 \end{bmatrix}$$